

ANTSDR을 위한 고급 **sysfs** 튜토리얼: Zynq SoC의 커널-유저스페이스 인터페이스 심층 실습

I. 아키텍처 심층 분석: **ANTSDR Zynq PS/PL**과 **Linux**의 유저스페이스 인터페이스

이 튜토리얼을 시작하기 전에, ANTSDR E200 장치의 핵심 아키텍처를 이해하는 것이 중요합니다. ANTSDR은 단순한 SDR이 아니라, Xilinx Zynq-7020 SoC(System on Chip)를 기반으로 하는 복잡한 임베디드 시스템입니다. Zynq 아키텍처는 두 개의 매우 다른 컴퓨팅 도메인을 단일 다이에 통합합니다.

A. Zynq-7020 SoC 아키텍처 이해

1. **PS (Processing System, 프로세싱 시스템):** 이것은 칩의 '두뇌'에 해당하며, 듀얼 코어 ARM Cortex-A9 CPU를 포함하는 하드-실리콘(hard-silicon)입니다. 이 PS는 ANTSDR의 임베디드 Linux 운영체제를 실행하며, 네트워킹, 스토리지 및 전반적인 시스템 관리를 담당합니다.
2. **PL (Programmable Logic, 프로그래머블 로직):** 이것은 Artix-7 FPGA 패브릭으로, '근육'에 해당합니다. PL은 소프트웨어로 처리하기에는 너무 빠르거나 집약적인 병렬 작업을 위해 하드웨어에서 직접 구현됩니다. ANTSDR에서 PL은 AD936x 트랜시버의 고속 샘플링 데이터를 처리하고, DMA(직접 메모리 액세스) 연결을 관리하며, 기타 맞춤형 디지털 신호 처리(DSP) 로직을 수행합니다.

PS(Linux)는 고속 AXI 버스를 통해 PL(FPGA)을 구성하고 제어합니다. 우리가 **sysfs**를 통해 상호작용할 때, 우리는 본질적으로 Linux 유저스페이스에서 PS를 통해 PL의 하드웨어 레지스터를 조작하게 됩니다.

B. sysfs: 하드웨어와 유저스페이스 간의 다리

sysfs는 Linux 커널이 제공하는 가상 파일 시스템으로, 일반적으로 /sys 디렉터리에 마운트됩니다. 이것은 /dev (원시 장치 노드 제공) 또는 /proc (프로세스 및 레거시 시스템 정보 제공)와는 다릅니다. sysfs의 핵심 목적은 커널의 내부 장치 모델 객체(kobjects), 그 속성 및 장치 간의 계층적 관계를 유저스페이스로 내보내는 것입니다.

ANTSDR에서 이는 ad9361-phy 드라이버나 xilinx-gpio 드라이버와 같은 하드웨어 드라이버가 커널에 등록될 때, /sys 내부에 파일을 생성하여 사용자가 셸(shell)을 통해 하드웨어를 모니터링하고 제어할 수 있게 함을 의미합니다.

C. PS 대 PL 주변장치의 중요성 구분

Zynq 아키텍처는 sysfs에서 상호작용할 때 중요한 구분을 만듭니다.

1. **PS 주변장치 (MIO):** Zynq PS는 자체적인 하드웨어 GPIO 컨트롤러를 포함하고 있으며, 이 핀들은 칩의 MIO(Multiplexed I/O)에 연결됩니다. Linux가 이들을 제어할 때, 드라이버는 PS 내부의 고정된 실리콘 레지스터와 직접 통신합니다.
2. **PL 주변장치 (EMIO):** ANTSDR E200의 8핀 확장 헤더 또는 AD936x 데이터 경로와 같은 주변장치는 FPGA 로직(PL) 내에 AXI GPIO 컨트롤러와 같은 IP 코어로 구현됩니다. 이들은 EMIO(Extended MIO)를 통해 PS에 연결됩니다.

이 아키텍처의 핵심을 이해하는 것은 Zynq 개발자에게 매우 중요합니다. 사용자가 PL GPIO를 제어하기 위해 sysfs에 쓸 때 발생하는 일련의 과정은 다음과 같습니다.

1. 사용자가 셸에 `echo 1 > /sys/class/gpio/gpio960/value` (가상의 번호)를 입력합니다.
2. PS에서 실행 중인 Linux 커널이 이 쓰기(write) 요청을 수신합니다.
3. Xilinx GPIO 드라이버는 이 요청을 특정 메모리 주소(예: 0x41200000)에 대한 쓰기 작업으로 변환합니다.
4. 이 주소는 일반 RAM이 아니라, FPGA 로직(PL) 내부에서 실행 중인 axi_gpio IP 코어에 메모리 맵핑된(MMIO) 주소입니다.
5. PL의 IP 코어가 이 쓰기 작업을 수신하고 ANTSDR 보드의 물리적 핀 전압을 변경합니다.

이 튜토리얼은 이 추상화 계층을 탐색하여, Zynq 시스템에서 문제가 발생했을 때 그것이 사용자 명령어의 오류인지, sysfs 권한 문제인지, 커널 드라이버 문제인지, 디바이스 트리(VII절 참조) 설정 오류인지, AXI 버스 연결 문제인지, 아니면 실제 외부 하드웨어의 문제인지를 진단할 수 있도록 합니다.

II. sysfs 파일 시스템: 원칙과 실제 탐색

sysfs는 체계적인 디렉터리 구조를 가지고 있습니다.

- `/sys/bus`: 장치들이 연결된 버스(예: `iio`, `spi`, `platform`)별로 정렬되어 있습니다.
- `/sys/class`: 장치들이 기능적 "클래스"(예: `gpio`, `leds`, `iio`)별로 정렬되어 있습니다.
- `/sys/devices`: 장치 계층 구조의 실제 모습을 보여주며, 다른 디렉터리들은 대부분 이곳의 항목들을 가리키는 심볼릭 링크를 포함합니다.

A. sysfs의 기본 메커니즘: show와 store

sysfs와의 모든 상호작용은 두 가지 기본 작업으로 귀결됩니다.

1. 읽기 (**Show**): 시스템 매개변수를 읽습니다. 사용자가 `cat /sys/class/leds/led0/brightness`를 실행하면, 커널은 `leds-gpio` 드라이버에 정의된 `show()` 함수를 호출합니다.¹ 이 함수는 하드웨어 상태(예: GPIO 레지스터)를 읽어 "0" 또는 "1"과 같은 문자열로 포맷하여 사용자의 셸로 반환합니다.¹
2. 쓰기 (**Store**): 매개변수에 씁니다. 사용자가 `echo "heartbeat" > /sys/class/leds/led0/trigger`를 실행하면, 커널은 드라이버의 `store()` 함수를 호출합니다.¹ 드라이버는 "heartbeat" 문자열을 받아 파싱한 후, 해당 LED의 하트비트 트리거를 활성화하는 것과 같은 특정 동작을 수행합니다.¹

이 튜토리얼을 진행하면서 중요한 점은 **sysfs의 안정성에 대한 경고**입니다. 특히 GPIO와 관련하여 `/sys/class/gpio` 인터페이스는 "deprecated"(더 이상 사용되지 않음) 및 "obsolete"(구식)로 간주됩니다. 이는 비효율적이고 여러 프로세스가 하나의 핀을 제어하려 할 때 충돌을 막을 수 없기 때문입니다.

그러나 이 경고는 오직 `/sys/class/gpio`에만 적용됩니다. `IIO(/sys/bus/iio/...)` 및 `LEDs(/sys/class/leds/...)`를 위한 sysfs 인터페이스는 현대적인 방식이며 해당 하위 시스템을 제어하는 표준적인 방법입니다. 이 튜토리얼에서는 이 이중성을 명확히 다룰 것입니다.

III. 시스템 접속 및 환경 설정

이제 실제 실습을 위해 ANTSDR의 임베디드 Linux 환경에 접속합니다.

A. ANTSDR에 연결하기

1. 펌웨어 및 부팅 모드: 이 튜토리얼은 Analog Devices의 IIO 프레임워크에 의존합니다. ANTSDR E200이 PlutoSDR 호환 모드로 부팅되었는지 확인해야 합니다. 장치 네트워크 포트 아래의 DIP 스위치를 **QSPI**로 설정하십시오. (SD 카드로 부팅하면 일반적으로 USRP(UHD) 펌웨어가 로드되며, 이는 이 튜토리얼의 IIO sysfs 실습과 호환되지 않습니다.)
2. 네트워크 연결: ANTSDR의 이더넷 포트를 컴퓨터 또는 네트워크 스위치에 연결합니다. 장치는 기본적으로 192.168.1.10 IP 주소를 가집니다.
3. 로그인: SSH 클라이언트를 사용하여 장치에 접속합니다.

Bash

```
ssh root@192.168.1.10
```

4. 자격 증명: Pluto (QSPI) 펌웨어의 기본 로그인 정보는 다음과 같습니다:

- 사용자: root

- 비밀번호: analog

B. 임베디드 환경 탐색

로그인하면, ANTSDR 웰컴 배너와 함께 Buildroot 기반의 Linux 프롬프트가 나타납니다. 실습에 필요한 libiio 및 libgpiod 명령줄 유틸리티가 펌웨어 이미지에 포함되어 있는지 확인합니다.

Bash

```
# 웰컴 배너가 Pluto 펌웨어를 확인합니다
```

```
# IIO 유틸리티 확인
```

```
which iio_info
```

```
which iio_attr
```

```
# GPIO 유틸리티 확인
```

```
which gpiodetect
```

```
which gpioset
```

위의 명령어들이 경로(예: /usr/bin/iio_info)를 반환하면 실습을 진행할 준비가 된 것입니다.

IV. AD936x 트랜시버 상호작용: 산업용 I/O (IIO) 프레임워크

ANTSDR의 핵심 SDR 기능(AD9361/AD9363 트랜시버)은 Linux IIO 하위 시스템을 통해 관리됩니다. IIO는 센서, ADC, DAC를 위한 표준 프레임워크입니다.

A. IIO sysfs 인터페이스의 구조

IIO 장치들은 /sys/bus/iio/devices/ 디렉터리 아래에 iio:deviceX 형태로 존재합니다. 어떤 장치가 무엇인지 식별하기 위해 name 파일을 읽습니다.

Bash

```
# 등록된 모든 IIO 장치의 이름 나열
cat /sys/bus/iio/devices/iio:device*/name
```

일반적으로 다음과 같은 장치들을 찾을 수 있습니다:

- **ad9361-phy**: AD936x 트랜시버의 메인 컨트롤러. RF 주파수, 게인 등 핵심 파라미터를 설정합니다.
- **cf-ad9361-lpc**: RX (수신) 데이터 경로를 위한 DMA 컨트롤러.
- **cf-ad9361-dds-core-lpc**: TX (송신) 데이터 경로 및 DDS(Direct Digital Synthesis) 컨트롤러.
- **xadc**: Zynq SoC 내부의 ADC로, 칩의 전압과 온도를 모니터링합니다.

B. 방법 1: 직접적인 **sysfs** 조작 (기본 방식)

IIO 드라이버는 체계적인 파일 이름 명명 규칙을 사용합니다: [방향]_[채널]_[속성].

- `in_voltage0_gain_control_mode`: 입력, 채널 0 (RX1), 게인 제어 모드 속성.
- `out_altvoltage1_frequency`: 출력, 대체 전압 채널 1 (TX LO), 주파수 속성.

이 방식은 `iio:deviceX`의 정확한 경로를 알아야 하므로 번거로울 수 있습니다.

Bash

```
# ad9361-phy 장치의 디렉터리 찾기 (예: iio:device1)
cat /sys/bus/iio/devices/iio:device1/name
# > ad9361-phy

# RX LO 주파수(out_altvoltage0) 읽기
cat /sys/bus/iio/devices/iio:device1/out_altvoltage0_frequency
# > 2400000000

# RX1 게인 제어 모드(in_voltage0) 읽기
cat /sys/bus/iio/devices/iio:device1/in_voltage0_gain_control_mode
# > slow_attack
```

C. 방법 2: libiio 유틸리티를 통한 추상화된 제어 (실용적 방식)

`libiio`는 장치 탐색과 경로 구성을 자동화하는 `iio_info` 및 `iio_attr` 같은 유용한 명령줄 도구를 제공합니다.

- **`iio_info`**: 전체 IIO 컨텍스트, 장치, 속성을 보고합니다.
- **`iio_attr`**: `iio:deviceX` 경로를 몰라도 속성을 읽고 쓸 수 있는 대상 지정 도구입니다.

D. 실습 1: `iio_attr`로 RF 파라미터 제어하기

이 실습에서는 `iio_attr`을 사용하여 `ANTSDR`의 RF 파라미터를 실시간으로 변경합니다.

1. RX (수신) LO 주파수 읽기:

`ad9361-phy` 드라이버는 RX LO를 `altvoltage0`에 매핑합니다.

Bash

```
# 'ad9361-phy' 장치의 'altvoltage0' (RX LO)의 'frequency' 속성 읽기
iio_attr -d ad9361-phy altvoltage0 frequency
# 예상 출력: 2400000000
```

2. TX (송신) LO 주파수 쓰기:
TX LO는 altvoltage1에 매핑 됩니다.

Bash

```
# 'altvoltage1' (TX LO)의 주파수를 2.45 GHz로 설정
iio_attr -d ad9361-phy altvoltage1 frequency 2450000000

# 변경 사항 확인
iio_attr -d ad9361-phy altvoltage1 frequency
# 예상 출력: 2450000000
```

3. RX 게인 모드 변경:
RX1 채널은 voltage0입니다. -i 플래그는 "입력 채널"을 지정합니다.

Bash

```
# RX1 ('voltage0')의 현재 게인 모드 읽기
iio_attr -i -c ad9361-phy voltage0 gain_control_mode
# 예상 출력: slow_attack (또는 manual, fast_attack, hybrid)

# 게인 모드를 'manual'로 설정
iio_attr -i -c ad9361-phy voltage0 gain_control_mode manual

# 수동 모드에서 게인을 30 dB로 설정
iio_attr -i -c ad9361-phy voltage0 hardwaregain 30
```

E. 실습 2: 칩 센서 읽기 (온도)

AD936x는 내부 다이(die) 온도를 측정하는 센서가 내장되어 있습니다. 이 센서는 ad9361-phy 드라이버에 의해 temp0 채널로 노출됩니다.

pluto_temp.sh 스크립트 분석을 통해, temp0의 input 속성에서 반환되는 값은 섭씨가 아니라 밀리섭씨(millidegrees Celsius) 단위임을 알 수 있습니다.

Bash

```
# 센서에서 밀리섭씨(millidegrees Celsius) 원시 값 읽기
TEMP_MILLICELSIUS=$(iio_attr -c ad9361-phy temp0 input)

# 원시 값 출력
echo "원시 값: $TEMP_MILLICELSIUS"

# awk를 사용하여 섭씨로 변환 (S310의 로직과 유사)
echo $TEMP_MILLICELSIUS | awk '{printf "AD936x 칩 온도: %.2f C\n", $1 / 1000}'
```

표 1: iio_attr (libiio) 대 직접적인 sysfs 경로 매핑

이 표는 실용적인 iio_attr 명령어와 이 명령어가 내부적으로 호출하는 원시 sysfs 파일 경로를 연결하여, 사용자가 요청한 sysfs의 근본적인 동작을 명확히 합니다.

논리적 기능	iio_attr 명령어 (libiio 추상화)	직접적인 sysfs 파일 경로 (커널 인터페이스)	비고
RX1 LO 주파수 (읽기)	iio_attr -d ad9361-phy altvoltage0 frequency	cat /sys/bus/iio/devices /iio:deviceX/out_alt voltage0_frequenc y	X는 장치 인덱스. altvoltage0가 RX LO에 해당합니다.
TX1 LO 주파수 (쓰기)	iio_attr -d ad9361-phy altvoltage1 frequency 2400M	echo 2400000000 > /sys/bus/iio/devices /iio:deviceX/out_alt	altvoltage1이 TX LO에 해당합니다.

		voltage1_frequency	
RX1 게인 모드 (쓰기)	iio_attr -i -c ad9361-phy voltage0 gain_control_mode manual	echo manual > /sys/bus/iio/devices /iio:deviceX/in_volta ge0_gain_control_ mode	-i는 입력, voltage0는 RX1을 의미합니다.
칩 온도 (읽기)	iio_attr -c ad9361-phy temp0 input	cat /sys/bus/iio/devices /iio:deviceX/in_tem p0_input	값은 밀리섭씨 (millidegre es Celsius) 단위입니다.

V. 하드웨어 제어 실습: 범용 입출력 (GPIO) 조작

GPIO 제어는 sysfs의 이중성을 보여주는 가장 완벽한 예입니다. 레거시(구식) 방식과 모던(현대식) 방식이 공존합니다.

A. ANTSDR E200 GPIO 매핑 분석

GPIO 핀을 제어하기 전에, Linux 커널이 인식하는 핀의 '번호'를 알아야 합니다. 이는 Zynq 플랫폼에서 가장 혼란스러운 부분입니다.

1. **PS (MIO) 핀:** Zynq PS 컨트롤러(zynq_gpio)는 Linux 커널에 의해 gpiochipX로 등록됩니다. 이 칩의 base 번호를 찾아야 합니다.

```
Bash
# PS GPIO 컨트롤러(칩) 찾기
cat /sys/class/gpio/gpiochip*/label | grep zynq_gpio
# 예: /sys/class/gpio/gpiochip906/label: zynq_gpio

# 베이스 번호 확인 (이 예에서는 906)
cat /sys/class/gpio/gpiochip906/base
# > 906
```

2. **PL (EMIO) 핀:** ANTSDR의 8개 확장 핀은 PL을 통해 EMIO로 연결됩니다. EMIO 핀은 MIO 핀(Zynq-7000의 경우 54개, 0-53) 뒤에 번호가 매겨집니다.

- 매핑 공식 (**S267** 기반): $\text{Linux_GPIO_번호} = \text{<PS_베이스>} + \text{<MIO_핀_개수>} + \text{<EMIO_핀_인덱스>}$
- 예시: EMIO 0 (첫 번째 PL 핀) = $906 + 54 + 0 = 960$
- 예시: EMIO 7 (여덟 번째 PL 핀) = $906 + 54 + 7 = 967$

이 계산은 복잡하고 펌웨어에 따라 달라질 수 있습니다. 이것이 바로 `/sys/class/gpio`가 더 이상 사용되지 않는 주된 이유입니다.

B. 방법 1 (레거시): `/sys/class/gpio` 인터페이스

이 방법은 `CONFIG_GPIO_SYSFS=y` 커널 옵션이 활성화되어 있어야 작동합니다. `sysfs`의 기본 `show/store` 메커니즘을 학습하기 위해 사용합니다.

실습 3: 외부 LED 깜박이기 (레거시 방식)

ANTSDR의 첫 번째 PL 핀(EMIO 0, 계산된 번호 960)에 LED를 연결했다고 가정합니다.

Bash

```
# 테스트할 GPIO 번호 (EMIO 0)
GPIO_NUM=960
```

```
# 1단계: 핀을 sysfs 유저스페이스로 내보내기(export)
```

```
echo $GPIO_NUM > /sys/class/gpio/export
```

```
# 2단계: 핀의 방향을 'out' (출력)으로 설정
```

```
# (실패 시, 핀이 이미 다른 드라이버(예: LED)에 의해 사용 중일 수 있음 - VI절 참조)
```

```
echo "out" > /sys/class/gpio/gpio${GPIO_NUM}/direction
```

```
# 3단계: 루프에서 핀 토글(toggle)
```

```
while true; do
```

```
    echo 1 > /sys/class/gpio/gpio${GPIO_NUM}/value # LED 켜기
```

```
    sleep 0.5
```

```
    echo 0 > /sys/class/gpio/gpio${GPIO_NUM}/value # LED 끄기
```

```
sleep 0.5  
done
```

```
# (Ctrl+C로 중지)
```

```
# 4단계: 정리 (핀을 sysfs에서 제거)
```

```
echo $GPIO_NUM > /sys/class/gpio/unexport
```

C. 방법 2 (현대식): libgpiod 및 /dev/gpiochipN

이것이 현재 권장되는 방식입니다. libgpiod 유틸리티는 복잡한 번호 계산 없이 gpiochip과 '라인 오프셋'을 직접 사용합니다.

1. GPIO 칩 탐지:

gpiodetect는 시스템의 모든 GPIO 컨트롤러를 나열합니다.

```
Bash
```

```
gpiodetect
```

```
# 예상 출력 (S210, S211, S212, S260과 유사):
```

```
# gpiochip0 [zynq_gpio] (118 lines)
```

```
# gpiochip1 [axi_gpio] (8 lines) <-- 또는 이와 같이 별도의 칩일 수 있음
```

2. 칩 정보 검사:

gpioinfo를 사용하여 칩의 각 라인(핀)에 대한 세부 정보를 봅니다.

```
Bash
```

```
# Zynq PS 칩(gpiochip0)의 모든 라인 정보 표시
```

```
gpioinfo gpiochip0
```

```
# 예상 출력 (S212, S237, S323과 유사):
```

```
#...
```

```
# line 53: "MIO_53" unused input active-high
```

```
# line 54: "EMIO_0" unused input active-high <-- 첫 번째 PL 핀
```

```
# line 55: "EMIO_1" unused input active-high
```

```
#...
```

```
# line 61: "EMIO_7"  unused  input active-high <-- 여덟 번째 PL 핀
```

실습 4: 버튼 읽기 (현대식 방식)

첫 번째 PL 핀(gpiochip0, 라인 오프셋 54)에 버튼이 연결되어 있다고 가정합니다.

Bash

```
# gpiochip0의 라인 54 값 읽기  
VALUE=$(gpioget gpiochip0 54)
```

```
echo "버튼 상태: $VALUE"
```

실습 5: 폴링(Polling) 없는 하드웨어 이벤트 대기

libgpiod의 가장 큰 장점은 CPU를 낭비하는 폴링 루프 대신, 인터럽트 기반의 이벤트 대기가 가능하다는 것입니다.

Bash

```
echo "버튼 누르기를 기다리는 중 (rising edge)..."
```

```
# --num-events=1 : 한 개의 이벤트만 기다림  
# --rising-edge : 상승 엣지(눌림)만 감지  
gpiomon --num-events=1 --rising-edge gpiochip0 54
```

```
echo "버튼 눌림 감지! 작업 실행..."
```

```
# (이 스크립트는 버튼이 눌릴 때까지 여기서 효율적으로 대기(block)합니다)
```

표 2: GPIO 제어: 레거시 (sysfs) 대 현대식 (libgpiod)

이 표는 구식이지만 교육적인 /sys/class/gpio 방식과 현대적이고 강력한 libgpiod 방식을 직접 비교합니다.

작업	레거시 방식 (/sys/class/gpio)	현대식 방식 (libgpiod 유틸리티)	비고
핀 식별	cat /sys/class/gpio/gpio chip*/base 등으로 복잡한 번호 계산 필요	gpiodetect 및 gpioinfo로 칩과 라인 오프셋을 명확히 식별	libgpiod는 불안정한 전역 ID 대신 (칩, 오프셋) 쌍을 사용합니다.
방향 설정	echo "out" > /sys/class/gpio/gpio X/direction	gpioset 실행 시 자동으로 처리됨	gpioset은 실행되는 동안 핀을 출력으로 요청합니다.
값 쓰기 (High)	echo 1 > /sys/class/gpio/gpio X/value	gpioset gpiochipY X=1	gpioset은 프로세스가 종료될 때까지 핀 상태를 유지합니다. 지속하려면 -m time 또는 -z (daemonize) 옵션을 사용해야 합니다.
값 읽기	val=\$(cat /sys/class/gpio/gpio X/value)	val=\$(gpioget gpiochipY X)	gpioget은 핀의 현재 값을 읽습니다.
이벤트 대기	비효율적인 셀 루프(polling) 필요. (예: while ;; do... sleep 0.1; done)	gpiomon --rising-edge gpiochipY X	gpiomon은 커널 이벤트를 기반으로 하므로(인터럽트), CPU 사용량이 거의 없습니다.

VI. 하드웨어 제어 실습: 온보드 LED 제어

Linux 커널은 LED를 위한 전용 **sysfs** 하위 시스템을 제공합니다. 이는 **GPIO**와 달리 구식이 아니며, **LED** 제어를 위한 올바른 방법입니다.

A. **/sys/class/leds** 하위 시스템 탐색

ANTSDR(Pluto) 펌웨어는 온보드 LED를 이 디렉터리에 등록합니다.

Bash

```
ls /sys/class/leds/  
# 예상 출력: "antsdr:blue:status", "antsdr:red:error" 등과 같이  
# 디바이스 트리에 정의된 이름
```

B. 실습 6: 수동 LED 제어 (brightness)

antsdr:blue:status라는 LED를 찾았다고 가정합니다.

Bash

```
# 이 LED의 경로를 변수에 저장  
LED_PATH="/sys/class/leds/antsdr:blue:status"  
  
# 최대 밝기 확인 (보통 1, PWM의 경우 255)  
cat ${LED_PATH}/max_brightness  
# > 1  
  
# LED 켜기  
echo 1 > ${LED_PATH}/brightness
```

```
# LED 끄기  
echo 0 > ${LED_PATH}/brightness
```

C. 실습 7: 커널 트리거로 LED 자동화 (trigger)

이것은 LED 하위 시스템의 가장 강력한 기능입니다.

1. 사용 가능한 트리거 확인:

trigger 파일을 읽으면 사용 가능한 모든 트리거가 나열되며, 현재 활성화된 트리거는 ``대괄호로 표시됩니다.

Bash

```
cat ${LED_PATH}/trigger  
# 예상 출력 (S83, S338, S340 기반):  
# [none] heartbeat timer default-on gpio...
```

2. "heartbeat" (심장박동) 트리거 설정:

이 트리거는 CPU 부하에 따라 LED를 심장박동처럼 깜박이게 합니다.

Bash

```
echo "heartbeat" > ${LED_PATH}/trigger  
# 이제 LED가 커널에 의해 자동으로 깜박입니다.
```

3. "timer" (타이머) 트리거 설정:

사용자 정의 가능한 깜박임을 제공합니다.

Bash

```
echo "timer" > ${LED_PATH}/trigger  
  
# 'timer'를 선택하면 새로운 sysfs 파일이 나타납니다 (delay_on, delay_off)
```

```
ls ${LED_PATH}/  
# > brightness delay_off delay_on device...
```

```
# 200ms 켜지고 800ms 꺼지도록 설정 (단위: ms)
```

```
echo 200 > ${LED_PATH}/delay_on
```

```
echo 800 > ${LED_PATH}/delay_off
```

4. 수동 제어로 복귀:

트리거를 **none**으로 설정하면 **brightness** 파일을 통해 다시 수동으로 제어할 수 있습니다.

Bash

```
echo "none" > ${LED_PATH}/trigger
```

D. 실습 8: sysfs 계층과 소유권 증명

대부분의 임베디드 LED는 단순히 GPIO 핀에 연결되어 있습니다. 그렇다면 **leds-gpio** 드라이버와 **libgpiod**가 동일한 핀을 두고 경쟁하지 않을까요?

커널은 "consumer"(소비자)라는 개념을 통해 이를 관리합니다. **leds-gpio** 드라이버가 로드되면 해당 GPIO 핀을 "소비"한다고 커널에 알립니다.

1. **gpioinfo**로 소유권 확인:

gpioinfo를 실행하여 어떤 핀이 LED 드라이버에 의해 사용 중인지 확인합니다.

Bash

```
gpioinfo
```

```
#...
```

```
# line 50: "MIO_50"  used  output active-high consumer="antsdr:blue:status"
```

2. 소유권 충돌 시연:

위의 결과는 **gpiochip0**의 50번 라인(가상)이 **antsdr:blue:status** LED에 의해 "소비"되었음을 증명합니다. **leds-gpio** 드라이버가 활성화된 상태에서 **libgpiod**로 제어를 시도합니다.

Bash

```
# 'heartbeat' 트리거를 활성화하여 LED 드라이버가 핀을 제어하도록 함  
echo "heartbeat" > /sys/class/leds/antsdr:blue:status/trigger
```

```
# gpioset으로 핀을 강제로 켜려고 시도 (실패하거나 무시됨)  
# 장치가 사용 중(Device or resource busy)이라는 오류가 발생할 수 있습니다.  
gpioset gpiochip0 50=1
```

3. 소유권 해제 및 제어권 획득:
LED 드라이버를 비활성화하여 핀을 해제한 다음 libgpiod로 제어합니다.

Bash

```
# 커널 트리거 해제  
echo "none" > /sys/class/leds/antsdr:blue:status/trigger
```

```
# 밝기를 0으로 설정하여 LED 드라이버가 핀을 활성 상태로 두지 않도록 함  
echo 0 > /sys/class/leds/antsdr:blue:status/brightness
```

```
# 이제 핀이 해제되었으므로 gpioset이 성공적으로 제어권을 획득할 수 있습니다.  
# --mode=wait : Enter 키를 누를 때까지 핀 상태를 유지  
gpioset --mode=wait gpiochip0 50=1
```

이 실습은 Linux 커널 내의 여러 sysfs 하위 시스템이 어떻게 상호작용하고 리소스를 관리하는지 보여주는 핵심적인 예입니다.

VII. 고급 분석: ANTSDR 디바이스 트리(Device Tree) 해부

지금까지 실습한 모든 sysfs 파일과 디렉터리는 시스템 부팅 시 자동으로 생성된 것입니다. 이는 Linux 커널이 ANTSDR의 하드웨어 구성을 알고 있었기 때문입니다. ARM 임베디드 시스템에서 이 정보는 ****디바이스 트리(Device Tree)****라 불리는 .dts (소스) 또는 .dtb (컴파일된 바이너리) 파일을 통해 커널에 전달됩니다.

MicroPhase의 펌웨어 GitHub 저장소(antsdr-fw-patch, antsdr-pynq 등)에는 ANTSDR을 위한

zynq-antsdre200.dtb 또는 pl.dts와 같은 디바이스 트리 파일이 포함되어 있습니다.²

A. 디바이스 트리 바인딩 해부

1. 실습 6 (LED)은 어떻게 생성되었는가?

디바이스 트리 내에 **gpio-leds** 호환 노드가 정의되어 있기 때문입니다.

DTS

```
/* 디바이스 트리(.dts) 파일의 예시 */
leds {
    /* 1. 커널에 "gpio-leds" 드라이버를 로드하도록 지시 */
    compatible = "gpio-leds";

    /* 2. LED 노드 정의 */
    status_led: led@0 {
        /* 3. 이것이 /sys/class/leds/ 디렉터리 이름이 됨 */
        label = "antsdr:blue:status";

        /* 4. PS GPIO 컨트롤러(&gpio0)의 50번 핀에 연결됨을 지정 */
        gpios = <&gpio0 50 0>;

        /* 5. 부팅 시 기본 트리거를 'heartbeat'로 설정 */
        linux,default-trigger = "heartbeat";
    };
};
```

2. 실습 4 (PL GPIO)는 어떻게 생성되었는가?

FPGA 디자인(Vivado)에서 생성된 **axi_gpio** IP 코어가 디바이스 트리에 정의되어 있기 때문입니다.

DTS

```
/* Vivado가 생성한 디바이스 트리(.dtsi)의 예시 */
axi_gpio_0: gpio@41200000 { /* 1. PS 메모리 맵 0x41200000에 장치 정의 */
```

```

/* 2. Xilinx GPIO 드라이버를 로드하도록 지시 */
compatible = "xlnx,xps-gpio-1.00.a";

/* 3. 이 장치를 /dev/gpiochipY로 등록하도록 지시 */
gpio-controller;

#gpio-cells = ;

/* 4. FPGA 디자인의 AXI 버스 주소 */
reg = < 0x41200000 0x10000 >;
};

```

이 분석은 I절에서 설명한 PS/PL 상호작용의 전체 그림을 완성합니다. sysfs 인터페이스는 FPGA 하드웨어 설계에서 시작되어 디바이스 트리에서 Linux에 정의되고, 커널 드라이버에 의해 로드되어 최종적으로 사용자에게 파일로 노출되는 긴 체인의 마지막 고리입니다.

VIII. 결론 및 향후 개발 방향

이 튜토리얼은 ANTSDR E200의 sysfs 인터페이스를 통한 하드웨어 상호작용을 심층적으로 다루었습니다. 세 가지 주요 제어 방법론을 실습했습니다.

1. 원시(**Raw**) **sysfs (cat/echo)**: show/store 메커니즘을 이해하는 데 교육적이며, IIO 및 LED와 같이 sysfs가 기본 인터페이스인 하위 시스템에 필수적입니다.
2. **libiio** 유틸리티 (**iio_attr** 등): IIO 하위 시스템과 상호작용하기 위한 선호되는 명령줄 추상화 계층입니다.
3. **libgpiod** 유틸리티 (**gpiocfg** 등): GPIO 제어를 위한 현대적이고 강력하며 선호되는 방법입니다. 구식 /sys/class/gpio를 대체하며, 이벤트 기반(gpiomon) 처리가 가능합니다.

여기서 다룬 셸(shell) 기반 실습은 시작점입니다. 실제 SDR 애플리케이션(srsRAN, openwifi 등)을 구축하려는 개발자는 다음 단계로 나아가야 합니다. 즉, libiio 및 libgpiod 라이브러리를 C, C++ 또는 Python 애플리케이션 내에서 직접 사용하여 런타임에 하드웨어 파라미터를 프로그래밍 방식으로 동적 재구성해야 합니다. 이 튜토리얼에서 배운 sysfs의 기본 원칙은 이러한 고급 개발의 근간이 될 것입니다.

참고 자료

1. Sysfs in Linux Kernel - Linux Device Driver Complete Tutorial, 11월 12, 2025에 액세스,
<https://embetronicx.com/tutorials/linux/device-drivers/sysfs-in-linux-kernel/>
2. MicroPhase/antsdr-pynq: pynq framework for antsdr - GitHub, 11월 12, 2025에

엑세스, <https://github.com/MicroPhase/antsdr-pynq>